# Deploying Azure Infrastructure Using Terraform Made Simple

Writing code to build your cloud infrastructure saves you from manual clicking and human mistakes. Mastering Terraform is the fastest way to become a confident cloud engineer.

**What you'll learn:**

1. Setting Up Your Terraform Provider
2. Creating Your Digital Home (Resource Group)
3. Building the Private Highway (VNet and Subnets)
4. Launching Your Virtual Machine

**Aman Raj Singh**
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

## Setting Up Your Terraform Provider

Before Terraform can talk to Azure, you need to tell it who you are and which cloud provider to use. Think of this as logging into your Azure account via code so Terraform has permission to build things for you.

```
provider "azurerm" {
  features {}
}
```

2

TIP 2 OF 4

## Creating Your Digital Home (Resource Group)

Every cloud resource needs a home. A Resource Group is simply a container that holds all related resources like your virtual machines, networks, and hard drives together in one organized place.

```
resource "azurerm_resource_group" "rg" {
  name     ="my-first-rg"
  location = "East US"
}
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

## Building the Private Highway (VNet and Subnets)

A Virtual Network gives your servers a secure private space to live. Subnets slice that space into smaller lanes so you can separate public-facing web servers from private databases for better security.

```
resource "azurerm_virtual_network" "vnet" {
  name                = "my-vnet"
  address_space       = ["10.0.0.0/16"]
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

## Launching Your Virtual Machine

This is where the magic happens. You define an Ubuntu or Windows server in code, attach it to your secure subnet, and watch Terraform spin up a real working computer in the cloud within seconds.

```
resource "azurerm_linux_virtual_machine" "vm" {
  name                = "my-vm"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  size                = "Standard_B1s"
  admin_username      = "azureuser"
}
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

# Key Takeaways

+ Always run terraform plan before terraform apply
+ Keep your state files secure and never share them publicly
+ Use descriptive names for all your cloud resources
+ Clean up unused resources using terraform destroy to save money
+ Break your large configuration files into smaller, manageable modules

## Watch Out For

! Hardcoding passwords and API keys directly inside your code files
! Forgetting to initialize your working directory with terraform init
! Modifying Azure resources manually in the portal while using Terraform

**Follow for daily Cloud & DevOps guides!**

linkedin.com/in/mramanrajsingh